

SchemaGuard — Project Report

Semantic Compliance and Drift Detection for LLM-Generated Structured Outputs

Author: Pragati Narotam **Course:** INFO 7375 — Prompt Engineering for Generative AI **University:**
Northeastern University **Date:** April 2026 **Repository:** github.com/PragatiAN1109/schema-guard-llm-validation

Table of Contents

1. Executive Summary
2. Problem Statement
3. System Architecture
4. Domains and Schemas
5. Semantic Rule Engine
6. Prompt Engineering
7. Synthetic Data Generation
8. Drift Detection
9. Confidence Scoring and Decision Routing
10. API and Demo Interface
11. Evaluation Results
12. Tech Stack
13. Future Work
14. References

1. Executive Summary

SchemaGuard is a validation and monitoring layer for LLM-generated structured outputs. It detects cases where JSON records pass schema validation but contain semantic contradictions, cross-field inconsistencies, or distribution drift. The system validates records across two domains — healthcare intake and financial loan applications — using a three-layer pipeline: structural validation, semantic rule checking, and confidence-based routing.

Key results on seed evaluation data:

- 100% classification accuracy across 16 labeled records (both domains)
- Precision 1.0, Recall 1.0, F1 1.0, zero false quarantine rate
- Clear confidence separation between valid (mean 1.0) and invalid (mean 0.70–0.76) records
- Successful detection of all simulated distribution shifts

Components delivered: Prompt engineering (7 template families), synthetic data generation (labeled seed datasets), schema validation, semantic rule engine (10 rules), drift detection (4 signal types), confidence scoring, decision routing, REST API, Streamlit UI, evaluation pipeline with metrics and charts, project website.

2. Problem Statement

LLMs generating structured JSON can produce output that passes schema validation but is semantically wrong. A healthcare record with a discharge date before the admission date is valid JSON with correct types — but it's logically impossible. A loan approval for 52x the applicant's income clears every type check. These silent failures propagate into databases and decisions without triggering alerts.

The gap: Most teams rely solely on JSON schema validation, which catches type errors and missing fields but misses logical contradictions entirely. There is no standard lightweight layer for catching these semantic failures in LLM outputs.

SchemaGuard fills this gap with deterministic, auditable, rule-based validation that complements schema checks, tracks output distributions over time, and routes each record to trusted, flagged, or quarantined status.

3. System Architecture

The system processes records through a multi-stage pipeline:

Single Record Flow: Input JSON → Structural Validation → Semantic Validation → Confidence Scoring → Decision Routing → Explanation → Audit Log

Batch Flow: Batch of records → Per-record pipeline → Aggregate statistics → Drift Detection → Drift Alerts

Module Responsibilities

Module	Responsibility
schemas/	JSON schema definitions per domain
rules/	Semantic rule registry + 10 cross-field rules
validator/	Pipeline orchestration, structural + semantic checks, batch processing, explanations, audit
drift/	Baseline profiling, distribution comparison (z-score, PSI, null-rate, violation-rate)
scoring/	Confidence score computation + decision routing
api/	FastAPI REST endpoints
ui/	Streamlit demo interface
evaluation/	Metrics, charts, test scripts
data_gen/	Prompt templates, seed data, generation scripts

4. Domains and Schemas

Healthcare Intake Records

Fields include: patient_id, name, date_of_birth, gender, admission_date, discharge_date, diagnosis_code (ICD-10), diagnosis_description, treating_physician, medication, procedure_code, patient_age, emergency_admission.

Cross-field dependencies: temporal ordering (admission after birth, discharge after admission), age consistency (stated age vs computed age from dates), age-appropriate diagnoses, medication plausibility.

Financial Loan Application Records

Fields include: application_id, applicant_name, date_of_birth, annual_income, employment_status, loan_amount, loan_purpose, credit_score, existing_debt, application_date, approval_date, approved_amount, property_value.

Cross-field dependencies: temporal ordering (approval after application), ratio constraints (loan-to-income, debt-to-income), employment length vs age, approved amount within requested.

Both schemas are defined as JSON Schema Draft 7 with type constraints, patterns, enums, and ranges.

5. Semantic Rule Engine

The rule engine uses a decorator-based registry pattern. Each rule is a Python function that takes a record dict and returns a structured result with rule_id, passed/failed, severity, affected fields, and a human-readable message.

Healthcare Rules (HC-001 – HC-005)

Rule	Check	Severity
HC-001	Patient age matches DOB vs admission date	Critical
HC-002	Admission date is after date of birth	Critical
HC-003	Discharge date is after admission date	Critical
HC-004	Diagnosis is age-appropriate	Warning
HC-005	Medication is plausible for diagnosis	Warning

Finance Rules (FN-001 – FN-005)

Rule	Check	Severity
FN-001	Approval date is after application date	Critical
FN-002	Loan-to-income ratio within 10x limit	Critical
FN-003	Existing debt-to-income ratio within 60%	Warning
FN-004	Employment length plausible for applicant age	Critical
FN-005	Approved amount does not exceed requested	Critical

All rules run per record. Failures are collected (not short-circuited) and attached to the validation result.

6. Prompt Engineering

Seven prompt template families were designed for synthetic data generation and explanation:

Valid record prompts: Include full schema, all cross-field constraints, and realistic parameter variants (age ranges, condition types, applicant profiles). Temperature 0.7–0.9 for diversity.

Invalid record prompts: Instruct the LLM to inject one specific semantic contradiction while keeping everything else valid. Each prompt targets a specific rule (HC-001 through FN-005) with configurable difficulty (easy/medium/hard).

Edge-case prompts: Target boundary conditions — newborns, same-day discharge, minimum-income applicants, just-turned-18 applicants. These should pass all rules.

Explanation prompts: Convert rule violations into human-readable summaries at validation time. Fill-in-the-blank structure grounded in actual validation results.

All prompts enforce JSON-only output with a triple-fence pattern: instruction → schema → reinforcement.

7. Synthetic Data Generation

Seed Dataset

Hand-written seed data: 8 records per domain (3 valid, 3 invalid, 2 edge-case). Each invalid record targets a specific semantic rule. Each record includes metadata: record_id, category, violated_rules, notes.

Dataset Specification

Target: 300 records per domain (60% valid, 25% invalid, 15% edge-case). Invalid records are distributed evenly across rules with difficulty levels. Output format: JSONL with full label metadata.

Generation Pipeline

The generate_dataset.py script provides CLI scaffolding for domain selection, category selection, prompt loading, LLM calling (provider-agnostic), output labeling, and JSONL storage. LLM provider integration is ready for plug-in.

8. Drift Detection

SchemaGuard monitors four drift signals:

Signal	Metric	Threshold
Numeric field shift	z-score of mean difference	> 1.5 std devs
Categorical distribution shift	Population Stability Index (PSI)	> 0.20
Null rate change	Absolute difference	> 15%
Violation frequency change	Absolute difference	> 10%

Baseline generation: Computes per-field statistics (mean, std, min/max for numeric; frequency distributions for categorical; null rates) from a reference set of valid records.

Drift detection: Compares a new batch against the baseline. Returns a structured report with per-field metrics, alert flags, and severity levels (high/medium/low).

Simulated drift test results:

- Healthcare: gender shift (PSI=27.0), emergency admission shift (PSI=27.6), null-rate shift detected
- Finance: income shift (z=4.13, HIGH), credit score shift (z=2.66, MEDIUM), categorical shift detected

9. Confidence Scoring and Decision Routing

Scoring Logic

Start at 1.0, subtract penalties:

- Structural failure → 0.0 immediately
- Critical semantic violation → -0.30 per violation
- Warning semantic violation → -0.12 per violation
- Info semantic violation → -0.05 per violation
- Drift alert (batch mode) → -0.03 per alert (max -0.15)
- No semantic rules evaluated → -0.05

Decision Routing

Decision	Condition
Trusted	Confidence >= 0.85 AND all checks pass
Flagged	0.50 <= confidence < 0.85 OR non-critical violations at high confidence
Quarantined	Confidence < 0.50 OR structural failure OR critical violation with sub-trusted confidence

Thresholds are configurable via environment variables.

10. API and Demo Interface

REST API (FastAPI)

Endpoints: POST /validate (single record), POST /batch-validate (batch + drift), POST /generate (sample records), GET /health. Domain aliases supported (healthcare, finance). CORS enabled. Swagger docs at /docs.

Streamlit UI

Three tabs: Single Record Validation (paste JSON, color-coded decision badge, confidence bar, violated rules), Batch Validation (file upload or paste, summary cards, per-record results, drift alerts), Sample Browser (browse seed records, one-click validate, generate baseline).

11. Evaluation Results

Classification Accuracy

Domain	Records	TP	FP	TN	FN	Accuracy
Healthcare	8	3	0	5	0	100%
Finance	8	3	0	5	0	100%
Combined	16	6	0	10	0	100%

Metrics

Metric	Healthcare	Finance
Precision	1.0	1.0
Recall	1.0	1.0
F1 Score	1.0	1.0
False Quarantine Rate	0.0	0.0
Mean Confidence (valid)	1.00	1.00
Mean Confidence (invalid)	0.76	0.70

Confidence Separation

Valid records score 1.0 (perfect). Invalid records score 0.70–0.76 depending on violation severity. The gap between valid and invalid means is 0.24–0.30, demonstrating clear separation.

Drift Detection

All simulated distribution shifts were successfully detected across both domains.

12. Tech Stack

Component	Technology	Rationale
Backend	Python + FastAPI	Async, auto-docs, Pydantic integration
Schema validation	jsonschema Draft 7	Standards-compliant, portable definitions
Semantic rules	Custom registry + decorators	Simple, deterministic, no DSL overhead
Drift detection	z-score + PSI	Interpretable, well-understood metrics
Confidence scoring	Severity-weighted penalties	Configurable, transparent
Demo UI	Streamlit	Fastest path to interactive demo
Storage	JSON + SQLite	Zero-config, clone-and-run
LLM integration	Provider-agnostic	OpenAI / Anthropic / local

13. Future Work

Lightweight RAG: Add a vector store (Chroma/FAISS) with domain rule documentation for grounded explanation generation. Architecturally ready — explanation prompts can be augmented with retrieved context.

Expanded domains: Add API configuration records, e-commerce inventory, or insurance claims as additional validation domains.

Production deployment: Containerize with Docker, add CI/CD, deploy to cloud with proper auth.

Automated feedback loops: Use validation failures to re-prompt the LLM with corrective instructions.

Larger evaluation datasets: Scale from 16 seed records to 300+ LLM-generated records per domain for statistical power.

14. References

1. JSON Schema Specification, Draft 7 — json-schema.org
2. Population Stability Index (PSI) — standard metric for distribution comparison in model monitoring
3. FastAPI Documentation — fastapi.tiangolo.com
4. Streamlit Documentation — docs.streamlit.io
5. ICD-10 Code Reference — [who.int/classifications/icd](https://www.who.int/classifications/icd)

SchemaGuard — Catching the errors that schema validation misses.